ПР4

Аутентификация, авторизация, валидация и обработка ошибок

Отчёт по практической работе №4

Проект Event Map App

Дисциплина: Разработка веб-приложений

Студент: Фамилия Имя, группа ____

Преподаватель: Самат Илияс Мырзабекулы

Алматы, 2026

Аннотация

Отчёт описывает практическую работу №4 проекта Event Map App: модель пользователя, регистрацию и вход, JWT-аутентификацию, ролевую авторизацию, проверку владельца объекта, валидацию и единый формат ошибок. Приведены результаты автотестов и ручного протокола проверки доступа.

Содержание

1. Цель работы
2. Модель пользователя
3. Регистрация
4. Вход и JWT
5. Защита маршрутов
6. Роли и матрица доступа
7. Проверка владельца и защита от mass assignment
8. Единый формат ошибок
9. Валидация
10. Тестирование
11. Ручной протокол проверки доступа
12. Выводы

1. Цель работы

Цель ПР4 — добавить в Event Map App сквозную аутентификацию, авторизацию, валидацию и единообразную обработку ошибок.

- реализовать модель пользователя с ролями user / organizer / admin;
- добавить регистрацию и вход с безопасным хранением паролей;
- выдавать и проверять JWT во всех трёх сервисах;
- ограничить операции по роли и владельцу объекта;
- привести ошибки к единому JSON-формату;
- проверить сценарии 401/403/400 автотестами и вручную.

2. Модель пользователя

Модель реализована в users-service/accounts/models.py, миграция — accounts/migrations/0001_initial.py.

Поле	Назначение	Ограничения
id	идентификатор	первичный ключ
username	логин	уникальный, 3–150 символов
email	email	уникальный, валидируется
password	хеш пароля	Argon2, открытый пароль не хранится
role	роль	user / organizer / admin, по умолчанию user
date_joined	дата регистрации	заполняется автоматически

Таблица 1 — Поля модели пользователя

Пароли хешируются Argon2PasswordHasher. Администратор создаётся только доверенной management-командой create_admin, а не через публичную регистрацию.

```
cd users-service
python manage.py create_admin --username admin --email admin@example.com --password
'StrongPass123'
python manage.py shell -c "from accounts.models import User;
u=User.objects.get(username='user1'); print(u.password[:20])"
# argon2$argon2id$v=19...
```

3. Регистрация

Запрос POST /auth/register:

```
{
  "username": "organizer1",
  "email": "organizer1@example.com",
  "password": "StrongPass123",
  "role": "organizer"
}
```

Проверки при регистрации:

- username: 3–150 символов, латиница, цифры, _, ., -;
- email: корректный формат и уникальность без учёта регистра;
- password: 8–128 символов, сохраняется только Argon2-хеш;
- role: публично разрешены только user и organizer; admin запрещён;
- дубликат username или email возвращает 409 Conflict.

Ответы: 201 — пользователь создан; 400 ValidationError — ошибки полей; 409 Conflict — дубликат.

4. Вход и JWT

POST /auth/login принимает username или email и пароль. При успехе возвращается access_token.

```
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {"id": 2, "username": "organizer1", "role": "organizer", ...}
}
```

JWT содержит:

- sub — идентификатор пользователя;
- username, email, role — данные для авторизации;
- iss = event-map-users-service;
- iat, exp, jti — время выпуска, срок действия и идентификатор токена.

Подпись HS256, секрет JWT_SECRET, срок жизни 60 минут (JWT_EXPIRES_MINUTES). Токен передаётся в заголовке Authorization: Bearer <access_token>. JWT выбран потому, что сервисы разделены и должны проверять пользователя локально, без общей сессии.

Ограничение: токен нельзя отозвать до истечения срока без blacklist по jti. Refresh-токены и отзыв вынесены в план PR6.

5. Защита маршрутов

Сервис	Защищённые маршруты	Механизм
users-service	GET /auth/me, GET /users	require_auth, затем require_role
events-service	POST /events, PUT/PATCH/DELETE /events/{id}	require_auth + require_role + owner-check
attendance-service	POST /attendance, GET /attendance/user/{id}	require_auth + проверка пользователя/роли

Таблица 2 — Защищённые маршруты

Порядок проверок:

1. нет, испорчен или истёк токен → 401;
2. токен валиден, но роль или владелец не подходят → 403;
3. ресурс не найден → 404;
4. данные не проходят валидацию → 400;
5. непредвиденная ошибка → 500 без stack trace.

6. Роли и матрица доступа

Действие	user	organizer	admin
Просмотр событий	да	да	да
Создание события	нет (403)	да	да
Изменение своего события	нет (403)	да	да
Изменение чужого события	нет (403)	нет (403)	да
Удаление своего события	нет (403)	да	да
Удаление чужого события	нет (403)	нет (403)	да
Отметка своего участия	да	да	да
Отметка за другого	нет (403)	нет (403)	да
Просмотр чужих отметок	нет (403)	нет (403)	да
GET /users	нет (403)	нет (403)	да

Таблица 3 — Матрица ролей

7. Проверка владельца и защита от mass assignment

Для организатора organizer_id при создании события принудительно берётся из sub проверенного JWT, а не из тела запроса. Admin может передать organizer_id явно. При изменении и удалении organizer_id события сравнивается с sub; при несовпадении возвращается 403.

Аналогично user_id в attendance-service берётся из JWT. Пользователь не может отметить участие за другого или прочитать чужие отметки. Это защищает от mass assignment и подмены идентификаторов.

8. Единый формат ошибок

Все сервисы возвращают ошибки в одном формате:

```
{
  "error": {
    "code": "Forbidden",
    "message": "Недостаточно прав для выполнения операции",
    "details": [
      {"field": "role", "message": "требуется organizer или admin"}
    ]
  }
}
```

HTTP	code	Когда
400	ValidationError	поля не прошли проверку
400	BadRequest	пустое тело или некорректный JSON
401	Unauthorized	нет заголовка Authorization
401	InvalidToken	неверная подпись/issuer
401	TokenExpired	истёк срок действия
401	InvalidCredentials	неверный логин или пароль
403	Forbidden	недостаточно роли или владельца
404	NotFound	ресурс не найден
405	MethodNotAllowed	метод не поддерживается
409	Conflict	дубликат пользователя
500	InternalServerError	необработанная ошибка

Таблица 4 — Коды ошибок

Реализация: users-service — accounts/errors.py и accounts/middleware.py; events-service и attendance-service — errors.py с ApiError, register_error_handlers и обработчиками HTTPException/Exception. Клиент не получает stack trace.

9. Валидация

Объект	Правила
username	3–150 символов, латиница, цифры, _, ., -
email	корректный формат, уникальность
password	8–128 символов
role при регистрации	только user или organizer
title события	2–100 символов
description	не более 2000 символов
lat / lng	-90..90 / -180..180
starts_at	ISO 8601
category	значение из справочника

Объект	Правила
limit / page	1..100 / ≥ 1

Таблица 5 — Правила валидации

10. Тестирование

Результаты автоматических проверок:

Сервис	Команда	Результат
users-service	python manage.py test	10 тестов, OK
events-service	python verify.py	31 проверка, 0 ошибок
attendance-service	python verify.py	10 проверок, 0 ошибок

Таблица 6 — Результаты автотестов

Покрытые сценарии:

- хеширование пароля Argon2 и отсутствие открытого пароля;
- дубликаты username/email → 409;
- некорректные email, password, role → 400;
- вход по username и email, неверный пароль → 401;
- GET /auth/me без токена → 401, с токеном → 200;
- GET /users для user → 403, для admin → 200;
- неизвестный маршрут и некорректный JSON → единый формат;
- необработанное исключение → 500 без stack trace.

11. Ручной протокол проверки доступа

Проверялась операция DELETE /events/{id} от имени трёх субъектов:

Субъект	Заголовок	Код	Вывод
без токена	нет	401	не аутентифицирован
user1	Bearer USER_TOKEN	403	роль не позволяет
admin	Bearer ADMIN_TOKEN	204	операция выполнена
organizer1 (владелец)	Bearer ORG_TOKEN	204	операция выполнена
organizer2 (чужой)	Bearer OTHER_TOKEN	403	нет прав на объект

Таблица 7 — Проверка доступа к DELETE /events/{id}

Дополнительно проверены:

Запрос	Код
POST /auth/register (organizer1, user1, organizer2)	201
POST /auth/register с дубликатом username	409
POST /auth/register с bad email, short password, role=admin	400
POST /auth/login с верным паролем	200
POST /auth/login с неверным паролем	401
GET /auth/me без токена	401
GET /auth/me с токеном	200
GET /users с токеном user1	403
GET /users с токеном admin	200

Таблица 8 — Дополнительные проверки

Полный протокол с curl-командами и ответами: docs/auth-check.md.

12. Выводы

В ПР4 реализована сквозная аутентификация и авторизация для трёх сервисов. Пароли хранятся только в виде Argon2-хеша, JWT проверяется локально по подписи, сроку и issuer, роли и владелец объекта проверяются отдельно. Ошибки приведены к единому формату, а сценарии 401/403/400 покрыты автотестами и ручным протоколом.

Остаточные ограничения: нет rate limiting и отзыва JWT, refresh-токены не реализованы, attendance-service хранит данные в памяти. Эти пункты вынесены в план следующих работ.

Использованные файлы

- users-service/accounts/models.py, migrations/0001_initial.py;
- users-service/accounts/auth.py, errors.py, middleware.py, views.py, urls.py, tests.py;
- users-service/accounts/management/commands/create_admin.py;
- events-service/auth.py, errors.py, api.py, services.py, validation.py, verify.py;
- attendance-service/auth.py, errors.py, app.py, verify.py;
- docs/auth.md, docs/auth-check.md.